CallIQ

GenAI-Powered Customer Support Quality Auditor

GitHub Repository: github.com/sirishachoppara-77/GenAI-Powered-Customer-SupportQuality-Auditor

Intern Name: Sirisha Choppara
Email: sirishasambasivarao7@gmail.com

1. Project Overview

1.1 Problem Statement

Traditional customer support quality assurance is a slow, manual, and inconsistent process. QA managers can only review a small fraction of interactions — typically 2–5% — meaning most calls and chats go completely unaudited. This creates several compounding problems:

- Manual review of a single call takes 20–30 minutes, making scale impossible.
- Scoring is subjective and inconsistent between different reviewers.
- Compliance violations go undetected until a complaint or legal issue surfaces.
- Feedback reaches agents too late to change behaviour on active cases.
- No systematic way to track improvement trends across agents or teams over time.

1.2 Solution

CallIQ is a GenAI-powered quality auditing platform that reviews customer support chats and calls, assigns quality scores, detects compliance violations, and suggests improvements in real time. By combining NLP, RAG pipelines, and speech-to-text transcription, the platform leverages LLMs to evaluate tone, empathy, compliance with scripts, and resolution effectiveness. Designed for enterprises, BPOs, and SaaS companies, the solution enhances customer experience, ensures compliance, and reduces manual QA workload.

1.3 Project Outcomes (from Project Statement)

- Automated analysis of chat and call transcripts — COMPLETED
- LLM-based scoring of quality metrics (empathy, resolution, compliance) — COMPLETED
- Real-time detection of compliance breaches — COMPLETED
- RAG-powered contextual feedback and improvement suggestions — COMPLETED
- Dashboards for supervisors with agent-wise analytics — COMPLETED

2. Requirements

2.1 Functional Requirements

Data Ingestion & Transcription

- Accept .txt transcript files for direct processing.
- Accept audio files: .mp3, .wav, .m4a, .ogg, .flac, .aac.
- Accept video files: .mp4, .mov, .avi, .mkv — audio extracted via moviepy.
- Transcribe audio to text using Deepgram nova-2 with speaker diarisation and smart formatting.
- Batch transcribe M4A files via CLI (call_transcriber.py) → call_summaries.json.
- Summarise chat logs via CLI (chat_summarizer.py) → *_summary.txt via Groq API.
- Fall back to mock transcription in demo mode when no API key is present.

Quality Scoring & Compliance Engine

- Score every call across 5 dimensions using LLM: Greeting Quality, Empathy, Problem Understanding, Resolution Clarity, Professionalism.
- Each dimension scored 0–5; total 0–25; grade A–F based on percentage.
- Detect customer sentiment: Positive, Neutral, Negative.
- Assess escalation risk: Low, Medium, High.
- Detect 18 compliance keywords via rule-based scanner.
- Identify specific compliance violations and improvement suggestions via LLM.

- Suggestions include: priority (High/Medium/Low), category, issue description, action, and example script.

RAG Pipeline for Contextual Audits

- Ingest .txt, .md, .pdf policy documents from a configurable directory.
- Chunk documents into 400-word overlapping segments (80-word overlap).
- Embed using OpenAI text-embedding-3-small, OpenRouter, or TF-IDF fallback (zero dependency).
- Store and retrieve vectors using FAISS IndexFlatIP or SimpleVectorStore fallback.
- Retrieve top-k most relevant policy chunks for any given transcript.
- Generate policy-grounded audit findings citing specific source documents.
- Ship 5 default policy documents covering greeting, empathy, data privacy, resolution, professionalism.

Dashboard & Visualization Hub

- Streamlit web UI with 5 tabs: Analyze, History & Trends, Policy RAG Audit, Alerts & Reports, About.
- Radar chart and bar chart for 5-dimension score visualisation.
- Score trend graph across all analysed calls (per agent and overall).
- Agent-wise analytics: filter by agent name, per-agent average score, top/bottom performers.
- Evaluation history persisted to CSV with export button.

Alerting & Reporting Module

- 5 configurable alert triggers: low score, high escalation, any violation, critical keywords, negative sentiment + low score.
- Email alerts via SMTP with full HTML template.
- Slack alerts via Block Kit webhook.
- Microsoft Teams alerts via Adaptive Card webhook.
- All alerts logged to calliq_alerts.json, viewable in the Alerts tab.
- Export audit reports as .txt, .pdf (ReportLab), and .xlsx (openpyxl with 4 sheets + bar chart).

2.2 Non-Functional Requirements

- Demo mode: full app functionality with no API keys using mock data.
- Graceful degradation: FAISS missing → SimpleVectorStore; no LLM → heuristic RAG audit.
- Cross-platform: Windows, macOS, Linux (tested on Windows PowerShell).
- Deployment-ready: Procfile for Render/Railway; .streamlit/config.toml for Streamlit Cloud.
- Secure: all API keys read from environment variables — no hardcoded secrets in source.
- MIT open-source license.

3. System Architecture

3.1 Architecture Overview

CallIQ is built as a modular pipeline. Each uploaded file flows through a series of processing layers — transcription, LLM evaluation, compliance checking, RAG auditing, alerting, and report generation — before results are presented in the Streamlit UI and persisted to disk. All layers degrade gracefully when external services are unavailable.

Layer	Components & Responsibilities
Input Layer	Streamlit file uploader · .txt, audio (.m4a .mp3 .wav .ogg .flac .aac), video (.mp4 .mov .avi .mkv)
Transcription	moviepy: extract audio from video · Deepgram nova-2 REST API: STT with diarisation · mock_transcription(): demo fallback
LLM Evaluation	EVAL_PROMPT (structured JSON schema) · GPT-4o-mini via OpenRouter · mock_evaluation(): demo fallback
Compliance Engine	detect_compliance_keywords(): 18-keyword rule scanner · LLM violation flags in eval prompt · Prioritised improvement suggestions
RAG Pipeline	rag_engine.py: chunk_text() · embed (OpenAI/OpenRouter/TF-IDF) · FAISSStore / SimpleVectorStore · rag_audit() · heuristic fallback
Alerting	alerts.py: evaluate_triggers() · Email (SMTP) · Slack (webhook) · Teams (webhook) · calliq_alerts.json log
Report Export	report_exporter.py: build_pdf_report() (ReportLab) · build_excel_report() (openpyxl, 4 sheets + bar chart)
Persistence	evaluations_history.csv · calliq_faiss.index + .pkl · policy_docs/ · calliq_alerts.json

3.2 File Structure

File / Folder	Purpose
app.py	Main Streamlit application — 5 tabs, full UI, transcription, evaluation, charts, session state (1,656 lines)
rag_engine.py	RAG pipeline module — ingestion, chunking, embedding, FAISS/SimpleVectorStore, retrieval, audit (606 lines)
alerts.py	Compliance alerting — 5 trigger types, Email/Slack/Teams dispatch, JSON log (356 lines)
report_exporter.py	PDF + Excel report generation — ReportLab, openpyxl, 4 Excel sheets (513 lines)
call_transcriber.py	CLI batch processor — M4A → Deepgram → OpenRouter summary → call_summaries.json
chat_summarizer.py	CLI chat summariser — Groq API with 4-model fallback chain → *_summary.txt
requirements.txt	All Python dependencies including reportlab, openpyxl, faiss-cpu, pypdf, moviepy==1.0.3
Procfile	Deployment config — streamlit run with --server.port=$PORT for Render/Railway
license.txt	MIT License — Copyright (c) 2026 Sirisha Choppara | Vidzai Digital
policy_docs/	Policy documents for RAG — 5 sample docs auto-created on first run
.streamlit/config.toml	Streamlit server + dark theme configuration
.gitignore	Excludes API keys, FAISS index, audio files, __pycache__, venv

4. Module-by-Module Breakdown

4.1 Transcription Module (app.py + call_transcriber.py + chat_summarizer.py)

The transcription layer handles all input types and converts them to plain text. It supports three entry points: the Streamlit UI (interactive), call_transcriber.py (batch M4A processing), and chat_summarizer.py (chat log summarisation).

transcribe_with_deepgram(): Sends raw audio bytes to Deepgram nova-2 REST API. Parameters: model=nova-2, smart_format=true, punctuate=true, diarize=true, language=en-US. Returns full transcript string.

extract_and_transcribe(): Orchestration function. Routes audio to Deepgram directly, routes video through moviepy to extract .wav first. Falls back to mock_transcription() on API error or missing key.

call_transcriber.py: CLI script for batch M4A files. Calls Deepgram for transcription, OpenRouter GPT-4 for two-line summary, writes call_summaries.json.

chat_summarizer.py: CLI script using Groq API with 4-model fallback chain (llama-3.1-8b → llama-3.3-70b → gemma2-9b → mixtral-8x7b). Outputs *_summary.txt.

4.2 Quality Scoring & Compliance Engine (app.py)

The evaluation engine sends the transcript to GPT-4o-mini via OpenRouter with a carefully engineered prompt demanding structured JSON output. Temperature is set to 0.2 for consistency.

EVAL_PROMPT: 50-line system prompt specifying exact JSON schema — 5 dimension scores, descriptions, total_score, customer_sentiment, escalation_risk, call_summary, agent_strengths, compliance_violations, improvement_suggestions (with category/priority/title/issue/action/example/dimension).

evaluate_with_openrouter(): Calls OpenRouter, strips markdown fences, parses JSON. Handles errors gracefully with fallback to mock_evaluation().

detect_compliance_keywords(): Scans transcript for 18 sensitive keywords: refund, lawsuit, legal action, sue, complaint, fraud, scam, cancel, unacceptable, terrible, horrible, never again, speak to manager, supervisor, escalate, data breach, privacy, discrimination, harassment.

mock_evaluation(): Deterministic demo evaluator using random.seed(len(transcript)) — same transcript always produces same result. Includes 4 realistic improvement suggestions in full structured format.

4.3 RAG Pipeline for Contextual Audits (rag_engine.py)

The RAG pipeline enables policy-grounded auditing — going beyond generic LLM knowledge to audit calls specifically against the company's own policy documents.

chunk_text(): Splits documents into 400-word chunks with 80-word overlap to preserve context at boundaries.

Embedding functions: Three-tier fallback — embed_texts_openai() (text-embedding-3-small), embed_texts_openrouter() (same model via OpenRouter), embed_texts_tfidf() (zero-dependency sparse vector, 384 dimensions, deterministic).

FAISSStore: Builds IndexFlatIP (inner product on L2-normalised vectors = cosine similarity). Saves to calliq_faiss.index + .pkl. Uses doc hash to skip rebuilding when unchanged.

SimpleVectorStore: Pure-Python fallback — cosine similarity over list of stored vectors. No external dependencies. Automatically used when faiss-cpu is not installed.

rag_audit(): Full RAG audit — retrieves top-6 policy chunks, builds context string, sends transcript + context to LLM, returns policy_references, policy_violations, policy_compliant_items, contextual_coaching, rag_summary.

_heuristic_rag_audit(): Offline fallback — checks 7 key phrases against retrieved chunks without any LLM call. Still provides actionable compliance feedback.

Sample policy docs: 5 documents auto-created in policy_docs/ on first run — greeting_policy.txt, empathy_compliance.txt, data_privacy_policy.txt, resolution_standards.txt, professionalism_standards.txt.

4.4 Dashboard & Visualization Hub (app.py — 5 Tabs)

Tab 1 — Analyze

File upload (drag-and-drop or browse), agent name input field, transcript preview, evaluation trigger. Results display: 5 metric cards (score/grade/sentiment/escalation/violations), dimension breakdown with expandable cards showing AI verdict and what is measured, radar + bar chart visualisation, call summary, agent strengths, compliance violations panel, compliance keywords panel, improvement suggestion cards with priority filter (All/High/Medium/Low), raw JSON expander, and three download buttons (.txt, .pdf, .xlsx).

Tab 2 — History & Trends

Overall metrics: calls analysed, average score, best/worst score, positive call count. Score trend line chart with per-dimension breakdown. Agent-wise analytics section: agent filter dropdown, per-agent average score, total calls per agent, bar chart comparing agents, top performer highlight, bottom performer alert. Full history dataframe with timestamp, filename, agent name, scores, sentiment, escalation. CSV export button.

Tab 3 — Policy RAG Audit

Left panel: policy library manager — upload .txt/.md/.pdf, view loaded docs with previews and line counts, index status badge (chunks indexed, store type), build/rebuild index button. Right panel: transcript source selector (manual paste or pick from last 5 session analyses), RAG audit runner. Results: compliance verdict banner, violations vs compliant items side by side, policy-grounded coaching, matched policy references with excerpts, raw retrieved chunks in expandable section.

Tab 4 — Alerts & Reports

Left panel: 5 configurable threshold controls (score slider, escalation checkbox, violation checkbox, negative sentiment checkbox, critical keywords text input). Three channel expanders — Email (SMTP host/port/user/password/from/to), Slack (webhook URL), Teams (webhook URL). Send Test Alert button dispatches a synthetic critical call to verify channels. Right panel: in-session alert display with severity badges, persistent alert history log (last 15 entries), alert log JSON export, export last report in .txt/.pdf/.xlsx.

Tab 5 — About

How it works (5 numbered steps), API configuration guide for all 4 keys, scoring guide with colour-coded 0–5 descriptions.

4.5 Alerting Module (alerts.py)

Fires automatically after every analysis. Five configurable triggers: (1) total score below threshold (default 10/25), (2) High escalation risk, (3) any compliance violation detected, (4) critical keywords found (lawsuit, fraud, data breach, etc.), (5) Negative sentiment combined with score below 15.

send_email_alert(): Sends styled HTML email via SMTP. Template matches CallIQ dark theme with score metrics, alert rows, severity colour coding.

send_slack_alert(): Sends Block Kit message with header, field grid (file/score/alerts/time), divider, and per-alert sections.

send_teams_alert(): Sends MessageCard Adaptive Card via incoming webhook with themeColor, summary, and fact list.

dispatch_alerts(): Master dispatcher — evaluates triggers, calls all enabled channels, logs every alert to calliq_alerts.json.

4.6 Report Export Module (report_exporter.py)

build_pdf_report(): Generates PDF using ReportLab SimpleDocTemplate. Includes: cover table (score/grade/sentiment/escalation colour-coded), dimension scores table with AI verdicts, agent strengths, compliance violations, improvement suggestion cards, full transcript on page 2. Responsible AI footer on every report.

build_excel_report(): Generates .xlsx using openpyxl with 4 sheets. Sheet 1 (Audit Summary): colour-coded metric tiles, call summary, dimension scores. Sheet 2 (Score Chart): bar chart auto-generated from dimension

data. Sheet 3 (Compliance & Suggestions): violations, keywords, structured suggestions table. Sheet 4 (Full Transcript): line-numbered transcript.

5. Development Phases & Milestones

The project was structured into 4 milestones as specified in the project statement, each delivered within its target 2-week window.

M#	Weeks	Objective	Key Deliverables	Status
M1	Weeks 1–2	**Setup & Transcription**	call_transcriber.py · chat_summarizer.py · Deepgram nova-2 STT · Groq fallback chain · call_summaries.json · demo transcript	**Completed**
M2	Weeks 3–4	**LLM Scoring Engine**	EVAL_PROMPT JSON schema · evaluate_with_openrouter() · mock_evaluation() · detect_compliance_keywords() · 5-dim scoring · grade A–F · sentiment · escalation	**Completed**
M3	Weeks 5–6	**RAG & Dashboards**	rag_engine.py · FAISSStore · SimpleVectorStore · TF-IDF embedder · Policy RAG Audit tab · History & Trends tab · agent-wise analytics · radar/bar/trend charts	**Completed**
M4	Weeks 7–8	**Alerts & Deployment**	alerts.py (Email/Slack/Teams) · report_exporter.py (PDF + Excel) · Procfile · .streamlit/config.toml · .gitignore · GitHub push · demo mode verified	**Completed**

5.1 Sprint-by-Sprint Detail

Sprint 1 — Environment Setup & API Research (Week 1)

- Python environment configured, VS Code set up, virtual environment created.
- GitHub repository initialised with README, .gitignore, MIT license.
- Deepgram, OpenRouter, Groq accounts created; API keys tested via Postman.
- Defined project folder structure and requirements.txt.

Sprint 2 — Transcription Pipeline (Week 2)

- call_transcriber.py built end-to-end: M4A → Deepgram → OpenRouter summary → JSON.
- Verified with real audio (audio_file.m4a — Martha's Florist call, 100% accurate transcript).
- chat_summarizer.py built with 4-model Groq fallback chain.
- call_summaries.json and human_chat_summary.txt produced as sample outputs.

Sprint 3 — LLM Scoring Engine (Week 3)

- EVAL_PROMPT engineered with structured JSON schema for consistent output.
- evaluate_with_openrouter() built with JSON cleanup and error handling.
- detect_compliance_keywords() implemented with 18-keyword dictionary.
- mock_evaluation() built for demo mode — deterministic, realistic output.

Sprint 4 — Streamlit UI (Week 4)

- app.py scaffolded with premium dark theme CSS (Syne + Inter fonts, #050508 background).
- Hero section, file uploader, analyze button, transcript preview built.

- Metric cards, dimension breakdown cards, radar chart, bar chart implemented.
- Improvement suggestion cards with expandable sections and priority filter.

Sprint 5 — RAG Pipeline (Week 5)

- rag_engine.py built: chunk_text(), TF-IDF embedder, SimpleVectorStore, ingest_policy_docs().
- FAISSStore added with IndexFlatIP, doc hash caching, save/load.
- 5 sample policy documents created covering all major compliance areas.
- End-to-end test: ingest → 5 chunks indexed → retrieve → heuristic audit → 4 compliant items.

Sprint 6 — RAG UI + History Tab (Week 6)

- Policy RAG Audit tab built with library manager and audit results display.
- History & Trends tab built with score trend chart and CSV persistence.
- Agent-wise analytics added: name input, per-agent filtering, comparative bar chart.

Sprint 7 — Alerts Module (Week 7)

- alerts.py built with 5 trigger types, Email/Slack/Teams dispatch, JSON logging.
- Alerts & Reports tab built with threshold sliders, channel config expanders, alert log.
- Auto-alert dispatch integrated into analysis pipeline — fires on every evaluation.
- Test Alert button added to verify channel configuration.

Sprint 8 — Reports + Deployment (Week 8)

- report_exporter.py built: PDF (ReportLab 4-section report) + Excel (4-sheet workbook with chart).
- Three download buttons integrated into Analyze tab (.txt / .pdf / .xlsx).
- Procfile, .streamlit/config.toml, .gitignore created for deployment.
- All API keys removed from source code; environment variable fallbacks added.
- Full project pushed to GitHub repository.

6. Testing

6.1 Unit Testing

Core logic tested via inline Python assertions:

- chunk_text(): 10 chunks from 200-word input with correct overlap — verified.
- embed_texts_tfidf(): 3 vectors, dim=384, L2 normalised — verified.
- SimpleVectorStore search: 'greet the customer warmly' scored 0.866 for 'welcome the caller' — verified.
- ensure_sample_policies(): 5 docs created in policy_docs/ — verified.
- _heuristic_rag_audit(): 3 compliant items, 0 violations on sample transcript — verified.
- evaluate_triggers(): 5 alerts fired on low-score/high-risk test result — verified.
- evaluate_triggers(): 0 alerts on clean high-score result — verified.
- build_pdf_report(): 4,401 bytes generated without errors — verified.
- build_excel_report(): 9,942 bytes, 4 sheets — verified

6.2 Integration Testing

Full pipeline tested with real data:

- audio_file.m4a → Deepgram → full transcript → call_summaries.json. Transcript accurate: customer name, email, address, product, price, delivery timeline all captured correctly.
- human_chat.txt → Groq API → human_chat_summary.txt generated successfully via chat_summarizer.py.
- RAG pipeline: 5 policy docs ingested → 5 chunks indexed → top-4 retrieved → heuristic audit → policy_compliant_items and policy_violations populated correctly.

- Alert dispatch: test result with score=6, escalation=High, violations present → 5 alert types triggered correctly.

6.3 Edge Cases & Handling

Edge Case	How Handled
No API keys configured	Full demo mode: mock_transcription() + mock_evaluation() + heuristic RAG — all 5 tabs functional
Deepgram API failure / timeout	try/except catches HTTP error; mock transcript returned; warning shown to user
LLM returns JSON in markdown fence	content.replace('```json','').replace('```','').strip() cleanup before json.loads()
FAISS not installed (Python 3.13)	SimpleVectorStore auto-selected — same API surface, pure Python, no install needed
moviepy 2.x API breaking change	moviepy==1.0.3 pinned in requirements.txt; documented in README
OpenRouter rate limit hit	except block falls to mock_evaluation(); user sees warning, not crash
Policy docs directory empty	ensure_sample_policies() auto-creates 5 default documents on first run
File above 50 MB uploaded	Warning shown; processing continues — no hard block
Doc hash unchanged on re-index	Cached FAISS index loaded from disk; skip rebuild unless force_rebuild=True
Negative shadow offset in pptxgen	Used angle:270 with positive offset to cast shadow upward — avoids file corruption
Agent name field left blank	Defaults to 'Unknown' in CSV; agent-wise chart skips Unknown if other agents present

7. Challenges & Solutions

#	Challenge	Solution	Outcome
1	moviepy 2.x broke imports — API paths changed from 1.x	Pinned moviepy==1.0.3 in requirements.txt; updated README	Video file transcription stable on all tested platforms
2	faiss-cpu wheel missing for Python 3.13 on Windows	Built SimpleVectorStore as full-featured pure-Python fallback	RAG works on Python 3.13 with zero additional installs
3	LLM returned JSON wrapped in markdown code fences	Post-processing: strip ` ``` `json and ` ``` ` before json.loads()	Zero parse errors after fix; robust across all model responses
4	matplotlib Agg backend error on Windows Streamlit	Added matplotlib.use('Agg') before import in app.py	Radar/bar/trend charts render correctly on Windows
5	TF-IDF fallback gave low similarity scores	Added _heuristic_rag_audit() as explicit phrase-matching offline mode	Offline users still get meaningful compliance feedback
6	Streamlit dark theme conflicting with custom CSS	Isolated all overrides in single <style> block; tested both themes	Consistent UI in both light and dark Streamlit modes
7	API keys hardcoded in scripts (security risk for GitHub)	Replaced all hardcoded keys with os.getenv() calls	No secrets in public repository; keys set via environment variables

8	Deepgram quota exhausted during development	Used mock_transcription() for all local dev; real API for final QA	Full development velocity maintained at zero API cost

8. System Outputs

8.1 Per-Call Evaluation Results

- Total score out of 25 with letter grade (A/B/C/D/F) and percentage.
- Individual scores 0–5 for all 5 dimensions with AI-generated verdict text.
- Customer sentiment: Positive / Neutral / Negative.
- Escalation risk: Low / Medium / High.
- Count and list of compliance violations.
- List of compliance keywords detected.
- 2-sentence call summary.
- 3 identified agent strengths.
- 3–5 improvement suggestions with priority, category, issue, action, and example script.

8.2 Visualisations

- Radar chart: spider plot showing all 5 dimensions simultaneously.
- Bar chart: colour-coded per dimension (green ≥4, amber ≥3, red <3).
- Score trend line: total score over all analysed calls with per-dimension dashed lines.
- Agent comparison bar chart: average score per agent with top/bottom performer highlighted.

8.3 Downloadable Reports

Format	Contents
.txt	Full structured text report — all scores, descriptions, violations, keywords, suggestions, full transcript
.pdf	Professional ReportLab PDF — cover metrics table, dimension scores, agent strengths, compliance, suggestion cards, transcript page
.xlsx	openpyxl workbook — 4 sheets: Audit Summary, Score Chart (bar chart), Compliance & Suggestions, Full Transcript (line-numbered)

8.4 RAG Audit Outputs

- Policy compliance verdict: 2-sentence overall assessment citing policy sources.
- Policy violations: specific breaches citing the source document and section.
- Policy-compliant actions: what the agent did correctly per policy.
- Policy-grounded coaching: 3–5 sentences tied directly to retrieved policy text.
- Matched policy references: source filename, relevant excerpt, relevance explanation.
- Retrieved chunks: top-k raw policy segments with relevance scores.

8.5 Alert Outputs

- In-app warning banner with severity level (critical / warning) after every triggered analysis.

- HTML email with score metrics, alert list, severity colour coding.
- Slack Block Kit message with field grid and per-alert sections.
- Teams Adaptive Card with themeColor and fact list.
- calliq_alerts.json: persistent log of all triggered alerts with timestamps.

8.6 Sample Output — audio_file.m4a (Martha's Florist)

The included audio_file.m4a is a real test recording of a flower ordering call. Randall Thomas orders one dozen long-stemmed red roses. Deepgram nova-2 accurately transcribed: customer full name (Randall Thomas), email (randall.thomas@gmail.com), phone (409-866-5088), shipping address (6800 Gladys Avenue, Beaumont, Texas 77706), product (12 long-stemmed red roses), total ($40), delivery commitment (24 hours). The call_summaries.json file in the repository demonstrates the complete pipeline output.

9. Deployment

9.1 Local Deployment (Windows)

1. Install Python 3.8+ from python.org — check Add Python to PATH during install.
2. Clone the repository: git clone https://github.com/sirishachoppara-77/GenAI-Powered-Customer-SupportQuality-Auditor.git
3. Install dependencies: pip install -r requirements.txt
4. Set API keys as environment variables (optional — app runs in demo mode without them):
   - $env:DEEPGRAM_API_KEY = 'your_key'
   - $env:OPENROUTER_API_KEY = 'your_key'
5. Run the app: python -m streamlit run app.py
6. Open http://localhost:8501 in your browser.

9.2 Streamlit Cloud (Recommended — Free)

7. Push repository to GitHub (already done).
8. Go to share.streamlit.io → New app.
9. Select the repository and set app.py as the entry point.
10. Add API keys under Settings → Secrets (TOML format).
11. Click Deploy — app is live at a public URL within 2 minutes.

9.3 Render.com

12. Connect GitHub repository to Render.com.
13. Create New Web Service — select Python environment.
14. The Procfile handles the start command: streamlit run app.py
15. Add API keys as Environment Variables in the Render dashboard.
16. Deploy — Render auto-builds and starts the service.

9.4 API Keys Reference

Key	Purpose	Model Used	Get It At
DEEPGRAM_API_KEY	Speech-to-text	nova-2	deepgram.com
OPENROUTER_API_KEY	LLM eval + RAG audit	openai/gpt-4o-mini	openrouter.ai
OPENAI_API_KEY	Embeddings (optional)	text-embedding-3-small	platform.openai.com
GROQ_API_KEY	Chat summarisation	llama-3.1-8b-instant	console.groq.com

10. Responsible AI Notice

CallIQ is an assistive platform designed to support — not replace — human judgment in quality assurance processes. The following principles govern its responsible use:

17. AI evaluation scores must never be the sole basis for employment, disciplinary, or compensation decisions. All AI-generated assessments must be reviewed by a qualified human supervisor before any action is taken.
18. LLM evaluation is probabilistic. The same transcript may receive slightly different scores on different runs. Base decisions on trends across multiple evaluations, not on a single score.
19. Compliance violation detection is not a legal audit. Flagged items indicate potential issues that require human review, not confirmed violations.
20. Agent identity and customer personal data should be redacted from transcripts before sending to external LLM APIs, where required by applicable data privacy regulations (GDPR, CCPA, etc.).
21. The system is built for transparency — all scores include explanations, all suggestions state the specific issue and recommended action, and raw evaluation JSON is always available for inspection.
22. No audio recordings or transcripts are stored by the app — only the evaluation metadata (scores, sentiment, escalation) is persisted to the history CSV.

11. Future Scope

11.1 Near-Term Enhancements

- Pinecone cloud vector store: replace FAISS with Pinecone for team-shared policy indexes accessible across devices.
- LangChain integration: replace manual RAG with LangChain chains for more sophisticated multi-hop retrieval.
- Real-time analysis: WebSocket-based live transcription and scoring during active calls using Deepgram streaming API.
- Multi-language support: Deepgram supports 30+ languages; extend evaluation prompt for non-English call centres.

11.2 Platform Expansion

- Role-based access control: supervisor vs agent views with different permission levels.
- Batch processing UI: upload and score multiple files at once with progress bar and summary report.
- Whisper integration: add OpenAI Whisper as an alternative STT option for offline/on-premise deployments.
- CRM integration: push evaluation results directly to Salesforce, HubSpot, or Zendesk agent profiles.
- Scheduled reports: weekly/monthly summary reports emailed automatically to supervisors.

11.3 Advanced AI Features

- Fine-tuned scoring model: train a domain-specific scoring model on historical evaluations for better accuracy.
- Trend prediction: use historical scores to predict agent performance trajectory and flag early burnout risk.
- Automated coaching: generate personalised training plans for agents based on their weakest dimensions.